



OpenViking 文档入库、分片、检索与排序流程说明

含架构图、流程图、分片规则、检索路径与排序算法说明

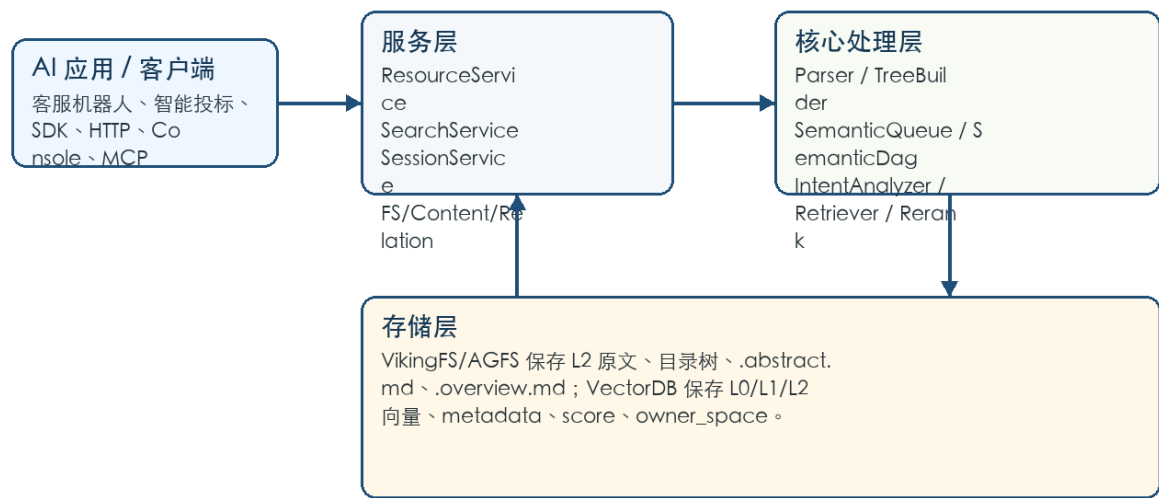
项目	内容
文档目的	通过图和步骤说明，让读者理解 OpenViking 从文档入库到检索排序的处理全过程。
核心范围	资源接入、文档解析、分片成树、L0/L1 语义加工、向量化、find/search、分层召回、rerank、最终排序。
阅读对象	评审专家、AI 应用研发、知识库实施、运维和交付人员。
版本口径	基于当前源码生成，日期 2026-04-27。

核心结论

OpenViking 的处理流程不是“上传后直接向量化”，而是先把资料拆成保持语义层级的知识树，再自底向上生成 L0/L1 摘要，最后在检索时通过意图分析、全局召回、层级递归、重排和热度融合完成排序。

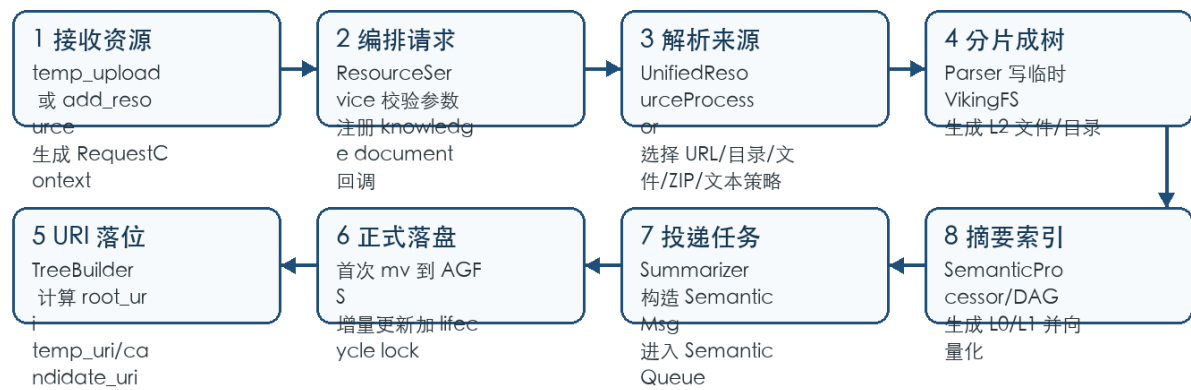
1. 架构图与总体流程

OpenViking 总体架构图



架构上，OpenViking 把上层 AI 应用、服务编排、核心处理管线和双层存储分离。文档入库、会话记忆和检索都依赖同一个 VikingFS/VectorDB 底座，因此客服机器人、智能投标、售前方案助手等应用可以复用同一套知识与上下文能力。

文档入库时序流程图



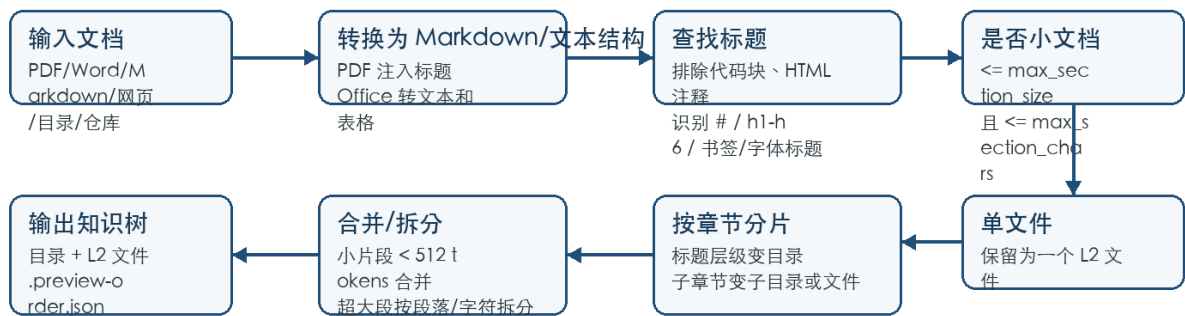
输出：root_uri、document_id、.abstract.md、.overview.md、L2 内容、向量索引记录、processing_status=ready/failed

1.1 处理链路摘要

阶段	关键模块	主要结果
接入	resources.py、ResourceService	合法 source、RequestContext、document 回调
解析	UnifiedResourceProcessor、ParserRegistry、Parser	临时 VikingFS 知识树、ParseResult
落位	TreeBuilder、ResourceProcessor	root_uri、temp_uri、正式 AGFS 目录
语义加工	Summarizer、SemanticQueue、SemanticProcessor、SemanticDag	.abstract.md、.overview.md、向量化任务
检索排序	VikingFS.find/search、IntentAnalyzer、HierarchicalRetriever、Rerank	MatchedContext 和最终 score

2. 分片机制详细说明

分片决策流程图



原则：OpenViking 优先保持文档自然层级，不做固定长度硬切；分片结果直接影响 L2 内容、L0/L1 摘要和检索路径。

OpenViking 的分片发生在 Parser 阶段。它不是按固定字符长度粗暴切块，而是先尽量把不同来源转换成 Markdown/文本结构，再根据标题层级、章节长度、段落边界和字符上限生成目录树与 L2 内容文件。TreeBuilder 不负责切分，只负责把 Parser 产生的临时树落到最终 URI。

判断条件	处理方式	输出形态
文档较小	估算 token <= max_section_size，且字符数 <= max_section_chars	保存为单个 L2 文件
无标题的大文档	按空行分段，累计 token/字符超过限制时切分	{文档名}_1.md、{文档名}_2.md...
有标题的大文档	识别最高层标题作为一级分片，保留标题层级	标题对应文件或目录
章节有子标题	父章节成为目录，直接内容作为虚拟 section，子标题继续递归	目录 + 子文件/子目录
小章节	小于 DEFAULT_MIN_SECTION_TOKENS，进入 pending，和相邻小节合并	merged 或 {first}_{count}more 文件
超大章节且无子标题	按段落切分；单段过长时按 max_section_chars 强制切	{章节名}_1.md、{章节名}_2.md...
文件名过长或重复	sanitize 后截断，并用 hash 后缀保证唯一	稳定、可落盘的 URI segment

2.1 分片配置与常量

配置/常量	当前口径	说明
ParserConfig.max_section_size	默认 1000 tokens	超过后触发分片；MarkdownParser 内部兜底常量为 1024。
DEFAULT_MIN_SECTION_TOKENS	512 tokens	小于该值的章节优先合并，避免碎片过小影响检索。
ParserConfig.max_section_chars	默认 6000 字符	硬性字符上限，防止 token 估算误差导致超大文件。
token 估算	CJK 字符约 0.7 token，其他非空白字符约 0.3 token	用于分片决策，不等同于模型 tokenizer 精确计数。
PREVIEW_ORDER_FILENAME	.preview-order.json	记录生成的 markdown_paths，便于预览和顺序恢复。

PDF/Office 如何进入分片

PDF 解析会尽量通过书签或字体分析识别标题，并把标题注入为 Markdown heading；Word、Excel、PowerPoint 等办公文档会先转换为可读文本/表格结构，再进入统一的分片与树构建逻辑。

3. 语义层与向量化

分片只产生 L2 内容树。真正用于检索的 L0/L1 摘要和向量索引由 SemanticProcessor/SemanticDag 异步生成。DAG 先处理叶子文件，再汇总子目录，最终自底向上生成父目录的 overview 和 abstract。

```
目录节点处理:
文件摘要 -> 子目录 abstract -> 生成 .overview.md
.overview.md -> 提取 .abstract.md
写入 VikingFS
vectorize_directory_meta:
  L0 Context(level=ABSTRACT, vectorize=abstract)
  L1 Context(level=OVERVIEW, vectorize=overview)

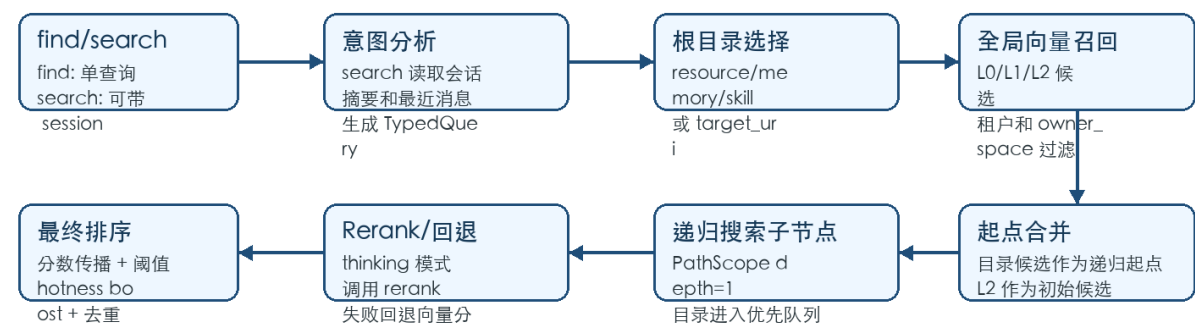
叶子文件处理:
_generate_single_file_summary -> summary_dict
vectorize_file:
  text 文件: summary_first / summary_only / raw content
  非文本文件: 使用 summary
  写入 L2 Context(level 默认 detail)
```

层级	物理位置/记录	向量化文本	检索作用
L0	每个目录的 .abstract.md 对应一条目录向量记录	短摘要 abstract	快速粗召回，定位相关知识区域
L1	每个目录的 .overview.md 对应一条目录向量记录	目录 overview	辅助目录导航和重排，理解子节点结构
L2	分片后的正文文件、代码文件、媒体摘要文件	summary 或原文截断内容	最终命中的证据片段和回答上下文

- 目录向量会写入 context_type、account_id、owner_space、level、parent_uri、active_count 等 metadata。
- 文本文件向量化默认优先使用摘要，避免长文档直接嵌入导致输入过长；必要时读取原文并按 max_input_chars 截断。
- EmbeddingTaskTracker 会追踪 SemanticMsg 关联的向量任务，全部完成后更新 knowledge document 状态。
- skip_vectorization=true 时只生成 L0/L1 文本，不写向量索引，适合只需要摘要不需要检索的场景。

4. 检索流程详细说明

检索与排序流程图



输出：MatchedContext(uri, level, abstract, score, context_type, relations)。L0/L1 URI 会补 .abstract.md 或 .overview.md 后缀

OpenViking 提供 find 和 search 两类语义检索。find 不依赖会话上下文，直接把用户 query 包装成 TypedQuery；search 可读取 session 的 latest_archive_overview 和最近消息，通过 IntentAnalyzer 生成多个 TypedQuery，并可同时检索 resource、memory、skill。

接口	是否使用会话	TypedQuery 来源	典型场景
find	否	直接构造单个 TypedQuery	知识库搜索框、快速定位资料、单轮问答
search	可选	有 session 时由 IntentAnalyzer 生成 0-5 条 TypedQuery；无 session 时按目标类型构造	多轮对话、智能投标、复杂客服问题
grep/glob	否	不走向量召回	精确文本查找、路径匹配、调试定位

4.1 检索步骤展开

检索步骤	输入	处理逻辑	输出
目标类型推断	target_uri 或 context_type	通过 URI 判断 resource/memory/skill；无 target_uri 时可搜索所有类型	TypedQuery.context_type
权限过滤	RequestContext	向量过滤 account_id、owner_space；resources 可账号共享，memory/skill 按用户或 agent 空间隔离	scope_filter
全局向量召回	query dense/sparse vector	search_global_roots_in_tenant 在 L0/L1/L2 中找候选	global_results
起点合并	root_uris + global_results	L0/L1 作为递归起点，L2 作为初始候选；thinking 模式可先 rerank 起点	starting_points、initial_candidates
递归子节点搜索	当前目录 URI	PathScope depth=1 搜索直接子节点；目录进入优先队列，L2 为终止命中	collected_by_uri
收敛判断	当前 topK URI 集合	topK 连续不变且达到 limit，最多 3 轮后停止	候选列表
结果转换	候选记录	读取 related_uri 的 L0 摘要，补 L0/L1 后缀，构造 MatchedContext	FindResult

5. 排序与打分规则

排序不是单一向量分数。OpenViking 在不同阶段会使用向量分数、rerank 分数、目录父级分数传播、阈值过滤和热度分数融合，最终返回 MatchedContext.score。

局部候选分数：

```
local_score = rerank_score if rerank 可用 且 mode=thinking else vector_score
```

递归传播分数：

```
final_score = alpha * local_score + (1 - alpha) * parent_score  
alpha = SCORE_PROPAGATION_ALPHA = 0.5
```

阈值过滤：

```
final_score > score_threshold  
或 score_gte=true 时 final_score >= score_threshold
```

最终展示分数：

```
display_score = (1 - HOTNESS_ALPHA) * semantic_score + HOTNESS_ALPHA * hotness_score  
HOTNESS_ALPHA = 0.2
```

排序因子	来源	作用
vector_score	向量库 dense/sparse/hybrid 搜索返回的 _score	基础相似度，用于全局召回和子节点召回。
rerank_score	RerankClient.rerank_batch(query, documents)	在 thinking 模式下重排候选摘要，提高语义贴合度。
parent_score	当前进入目录的分数	通过分数传播让高相关目录下的子节点获得合理加权。
score_threshold	调用参数或 rerank_config.threshold	过滤低分候选，控制噪声。
hotness_score	active_count + updated_at	常用、近期更新的上下文获得轻微提升。
dedup by URI	collected_by_uri	同一 URI 多次命中时保留最高 final_score。
relations	VikingFS relation graph	最终结果带最多 5 个相关节点摘要，扩展上下文。

Rerank 失败时怎么办

如果 rerank 服务异常、返回长度不一致或未配置，系统会记录 warning，并回退到向量分数。这样检索质量可能下降，但服务不会因为重排失败而不可用。

5.1 为什么要分层排序

为什么要分层排序	解释
避免只命中孤立碎片	L0/L1 先定位相关知识区域，再进入子节点，能保留文档结构和上下文关系。
降低大库搜索成本	全局召回只选起点，递归搜索只展开高分目录，减少无关节点遍历。
改善答案可解释性	MatchedContext 保留 level、uri、abstract、relations，方便追溯来源。
支持复杂任务	search 可把一个问题拆成多个 TypedQuery，分别检索资源、记忆和技能。

6. 端到端示例

示例: 上传“智能投标产品手册.docx”并问“这份材料里有没有酒店行业案例？”

入库:

1. temp_upload -> temp_file_id
2. add_resource(wait=true, instruction=面向投标提取产品能力、案例、资质)
3. WordParser 转换内容，MarkdownParser 分片成章节树
4. TreeBuilder 生成 root_uri=viking://resources/投标资料/智能投标产品手册
5. SemanticDag 生成每个目录的 .abstract.md/.overview.md
6. VectorDB 写入 L0/L1/L2 向量记录

检索:

1. search(query, session_id) 读取会话摘要和最近消息
2. IntentAnalyzer 生成 resource 类型 TypedQuery
3. 全局召回命中“行业案例”相关 L0/L1
4. 递归进入该目录，召回酒店案例 L2 分片
5. rerank 提升与“酒店行业案例”最相关的分片
6. 返回 MatchedContext，供 RAG 生成答案并引用来源

跟踪对象	产生阶段	用途
root_uri	TreeBuilder / ResourceProcessor	正式知识树入口，检索时可作为 target_uri。
分片文件 URI	MarkdownParser/Parser	L2 内容节点，最终答案引用来源。
.abstract.md	SemanticDag	L0 快速召回文本。
.overview.md	SemanticDag	L1 目录导航和重排文本。
SemanticMsg.id	Summarizer/SemanticQueue	关联语义处理和 embedding 任务。
document_id	KnowledgeDocumentRegistry	查询 ready/failed 处理状态。
MatchedContext.score	HierarchicalRetriever	最终排序后的可解释检索分数。

7. 源码映射

主题	关键源码	说明
资源接入	openviking/server/routers/resources.py	temp_upload、add_resource 路由和请求参数。
入库编排	openviking/service/resource_service.py	URI 约束、文档登记、wait、watch task。
执行处理	openviking/utlis/resource_processor.py	解析、TreeBuilder、正式落盘、Summarizer 投递。
来源路由	openviking/utlis/media_processor.py	URL、目录、文件、ZIP、原始文本路由。
分片核心	openviking/parse/parsers/markdown.py	标题识别、小节合并、超大段落拆分、preview-order。
解析器选择	openviking/parse/registry.py	根据扩展名、URL 类型选择 parser。
语义 DAG	openviking/storage/queuefs/semantic_dag.py	自底向上生成 L0/L1 并调度向量化。
向量化	openviking/utlis/embedding_utils.py	目录 L0/L1 和文件 L2 的 Context/EmbeddingMsg。
检索入口	openviking/storage/viking_fs.py	find/search、IntentAnalyzer 调用和结果聚合。
意图分析	openviking/retrieve/intent_analyzer.py	会话上下文转 TypedQuery。
分层召回排序	openviking/retrieve/hierarchical_retriever.py	全局召回、递归搜索、rerank、分数传播、hotness。
向量过滤	openviking/storage/ viking_vector_index_backend.py	tenant filter、PathScope、target_directories、owner_space。